

Projet de Systèmes et Réseaux

Jeu "6 qui prend" en ligne



Binôme :

LAWSON Ryan - MORAUX Thomas

Licence 3 Informatique
Université Bourgogne Europe

15 décembre 2024

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Présentation du jeu	3
2	Architecture générale du système	3
2.1	Modèle client-serveur	3
2.2	Processus impliqués	3
2.3	Isolation des processus	3
3	Mécanismes de communication	4
3.1	Tubes nommés (FIFOs)	4
3.1.1	Implémentation	5
3.1.2	Structure des messages	5
3.2	Sockets TCP/IP	6
3.2.1	Implémentation serveur	6
3.2.2	Gestion multi-clients	6
4	Synchronisation et gestion du temps	7
4.1	Sémaphores System V	7
4.2	Opérations sur les sémaphores	7
4.3	Gestion des timeouts	7
5	Fonctionnalités principales	8
5.1	Gestionnaire du jeu	8
5.2	Interface joueur humain	8
5.3	Joueurs robots	9

6	Structure de données	10
6.1	Représentation des cartes	10
6.2	Calcul des têtes de bœuf	10
6.3	Gestion des parties	11
7	Journalisation et statistiques	11
7.1	Système de logging	11
7.2	Exemple de log	11
7.3	Génération de statistiques	12
8	Tests et validation	12
8.1	Stratégie de test	12
8.2	Tests unitaires	13
9	Manuel d'utilisation	13
9.1	Compilation	13
9.2	Commandes disponibles	14
9.3	Exemples d'utilisation	14
10	Discussion et améliorations	14
10.1	Difficultés rencontrées	14
10.2	Contribution de l'IA générative	15
11	Conclusion	15
11.1	Bilan technique	15
11.2	Perspectives	15
11.3	Code source	15

1. Introduction

1.1. Contexte du projet

Le projet "6 qui prend" s'inscrit dans le cadre du module de Systèmes et Réseaux de la Licence 3 Informatique. L'objectif est de développer une application permettant à plusieurs joueurs de participer à une partie du célèbre jeu de cartes de Wolfgang Kramer (1994) en utilisant les mécanismes de communication inter-processus et les sockets réseau.

1.2. Présentation du jeu

"6 qui prend" est un jeu de cartes stratégique où chaque joueur cherche à accumuler le moins possible de "têtes de bœuf". Le jeu se déroule en plusieurs manches avec des règles spécifiques concernant le placement des cartes sur des rangées et la prise des cartes lorsque la sixième carte est posée.

2. Architecture générale du système

2.1. Modèle client-serveur

Notre application suit une architecture maître-esclave où le **Game Manager** agit comme processus coordinateur central. Ce choix architectural permet une gestion centralisée des parties tout en maintenant l'indépendance des joueurs.

2.2. Processus impliqués

- **Gestionnaire du jeu** : Processus maître orchestrant les parties
- **Joueurs humains** : Processus esclaves avec interface utilisateur
- **Joueurs robots** : Processus autonomes avec stratégies pré-programmées
- **Client réseau** : Interface pour joueurs distants

2.3. Isolation des processus

Chaque joueur est exécuté dans un processus séparé créé via `fork()` et `exec()`. Cette isolation offre plusieurs avantages :

- Séparation mémoire garantissant la sécurité
- Gestion indépendante des erreurs
- Scalabilité facile
- Possibilité d'exécution sur différentes machines

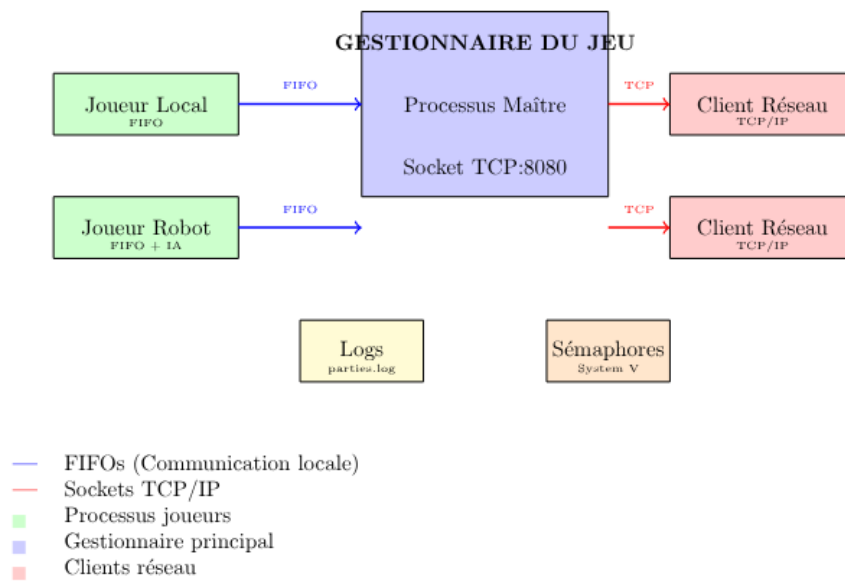


Figure 1: Architecture du système "6 qui prend"

FIGURE 1 – Architecture générale du système

3. Mécanismes de communication

3.1. Tubes nommés (FIFOs)

Nous utilisons des FIFOs pour la communication locale entre le gestionnaire et les joueurs locaux.

3.1.1. Implémentation

```
1 int creer_fifo(const char* chemin) {
2     if (mkfifo(chemin, 0666) == -1) {
3         if (errno != EEXIST) {
4             perror("[FIFO] Erreur création");
5             return -1;
6         }
7     }
8     printf("[FIFO] Cr      : %s\n", chemin);
9     return 0;
10 }
11
12 int envoyer_fifo(int fd, const void* data, size_t taille) {
13     ssize_t bytes_ecrits = write(fd, data, taille);
14     if (bytes_ecrits != (ssize_t)taille) {
15         perror("[FIFO] Erreur écriture ");
16         return 0;
17     }
18     return 1;
19 }
```

Listing 1 – Fonctions FIFOs dans communication.c

3.1.2. Structure des messages

```
1 typedef struct {
2     int type;
3     int id_joueur;
4     int id_partie;
5     union {
6         Carte carte;
7         int score;
8         int choix;
9         char message[256];
10    } donnees;
11 } Message;
12
13 enum TypeMessage {
14     MSG_DEMANDE_CARTE = 1,
15     MSG_CARTE_JOUE = 2,
16     MSG_INFO_PARTIE = 3,
17     MSG_SCORE = 4,
18     MSG_FIN_MANCHE = 5,
19     MSG_FIN_PARTIE = 6,
20     MSG_ERREUR = 7,
21     MSG_PING = 8,
22     MSG_CONFIRMATION = 9
23 };
```

Listing 2 – Structure Message dans joueurs.h

3.2. Sockets TCP/IP

Pour la communication réseau, nous utilisons des sockets TCP avec le protocole suivant :

Étape	Description
Connexion	Client se connecte au port 8080
Handshake	Envoi du message de bienvenue
Communication	Échange de messages formatés
Déconnexion	Fermeture propre des sockets

TABLE 1 – Protocole de communication TCP

3.2.1. Implémentation serveur

```

1 int creer_serveur_tcp(int port) {
2     int sockfd = socket(AF_INET, SOCK_STREAM, 0);
3     struct sockaddr_in serveur_addr;
4
5     serveur_addr.sin_family = AF_INET;
6     serveur_addr.sin_addr.s_addr = INADDR_ANY;
7     serveur_addr.sin_port = htons(port);
8
9     bind(sockfd, (struct sockaddr*)&serveur_addr, sizeof(
10         serveur_addr));
11     listen(sockfd, BACKLOG_TCP);
12
13     return sockfd;
14 }
```

Listing 3 – Création du serveur TCP

3.2.2. Gestion multi-clients

```

1 int surveiller_sockets_tcp(int sock_serveur, ClientTCP clients[],
2     int* nb_clients, int max_clients) {
3     fd_set readfds;
4     FD_ZERO(&readfds);
5     FD_SET(sock_serveur, &readfds);
6     // Ajouter les clients existants
7     for (int i = 0; i < *nb_clients; i++) {
8         if (clients[i].est_actif) {
9             FD_SET(clients[i].socket, &readfds);
10        }
11    }
12    struct timeval timeout = {0, 100000}; // 100ms
13    return select(max_fd + 1, &readfds, NULL, NULL, &timeout);
14 }
```

Listing 4 – Surveillance des sockets avec select()

4. Synchronisation et gestion du temps

4.1. Sémaphores System V

Pour garantir l'accès exclusif aux ressources partagées, nous utilisons des sémaphores IPC System V.

```
1 SemaphoreJeu* creer_semaphores(int nb_sem, int valeur_init) {
2     key_t key = CLE_SEM;
3     int sem_id = semget(key, nb_sem, IPC_CREAT | IPC_EXCL | 0666);
4
5     if (sem_id == -1 && errno == EEXIST) {
6         sem_id = semget(key, nb_sem, 0);
7     }
8
9     // Initialisation des s maphores
10    union semun arg;
11    arg.val = valeur_init;
12
13    for (int i = 0; i < nb_sem; i++) {
14        semctl(sem_id, i, SETVAL, arg);
15    }
16
17    SemaphoreJeu* sem = malloc(sizeof(SemaphoreJeu));
18    sem->sem_id = sem_id;
19    sem->nb_sem = nb_sem;
20    return sem;
21 }
```

Listing 5 – Implémentation des sémaphores

4.2. Opérations sur les sémaphores

- `sem_wait_jeu()` : Opération P (wait)
- `sem_signal_jeu()` : Opération V (signal)
- `sem_getvalue()` : Lecture de la valeur courante

4.3. Gestion des timeouts

Chaque joueur dispose de 30 secondes pour jouer son coup. Ce mécanisme est implémenté avec `alarm()` et le signal `SIGALRM`.


```

1 void gestionnaire_timeout(int sig) {
2     if (sig == SIGALRM) {
3         timeout_atteint = 1;
4         printf("      ␣Timeout!␣Le␣joueur␣a␣mis␣trop␣de␣temps.␣\n");
5     }
6 }
7
8 int configurer_timeout(int secondes) {
9     timeout_atteint = 0;
10    signal(SIGALRM, gestionnaire_timeout);
11    alarm(secondes);
12    return 0;
13 }

```

Listing 6 – Gestion du timeout

5. Fonctionnalités principales

5.1. Gestionnaire du jeu

Le Game Manager est le cœur du système avec les responsabilités suivantes :

Fonction	Description
Accueil joueurs	Gestion des connexions entrantes
Distribution cartes	Mélange et distribution aléatoire
Validation coups	Vérification des règles du jeu
Calcul scores	Mise à jour des scores en temps réel
Logging	Enregistrement des événements dans un fichier

TABLE 2 – Fonctions du Game Manager

5.2. Interface joueur humain

L'interface texte permet aux joueurs humains d'interagir avec le jeu :

```

1 int demander_choix_carte_humain(Joueur* j) {
2     printf("\n===␣VOTRE␣MAIN␣===\n");
3     for (int i = 0; i < j->nb_cartes_main; i++) {
4         printf("%2d.␣", i + 1);
5         afficher_carte(j->main[i]);
6         printf("\n");
7     }
8
9     int choix = -1;
10    while (choix < 1 || choix > j->nb_cartes_main) {

```

```
11     printf("\nChoisissez une carte (1-%d):", j->
12           nb_cartes_main);
13     // Lecture et validation de l'entr e
14 }
15 return choix - 1;
```

Listing 7 – Interface joueur humain

5.3. Joueurs robots

Trois niveaux de stratégie ont été implémentés pour les robots :

1. **Niveau débutant** : Joue toujours la plus petite carte
2. **Niveau intermédiaire** : Essaie d'éviter de prendre des rangées
3. **Niveau avancé** : Stratégie agressive pour faire perdre les autres

```
1 Carte strategie_agressive(Joueur* robot, const Rangee rangees[],
2   int nb_rangees) {
3     // Recherche des rang es presque pleines
4     int rangees_presque_pleines[4] = {0};
5     for (int i = 0; i < nb_rangees; i++) {
6         if (rangees[i].nb_cartes == 5) {
7             rangees_presque_pleines[i] = 1;
8         }
9     }
10    // Chercher une carte qui peut tre plac e sur une rang e
11    // presque pleine
12    for (int i = 0; i < robot->nb_cartes_main; i++) {
13        Carte carte = robot->main[i];
14        for (int r = 0; r < nb_rangees; r++) {
15            if (rangees[r].nb_cartes > 0 &&
16                carte.valeur > rangees[r].cartes[rangees[r].
17                  nb_cartes-1].valeur &&
18                  rangees_presque_pleines[r]) {
19                return carte;
20            }
21        }
22    }
23    // Sinon, jouer la plus grande carte
24    return robot->main[robot->nb_cartes_main - 1];
25 }
```

Listing 8 – Stratégie du robot avancé

6. Structure de données

6.1. Représentation des cartes

```
1 typedef struct {
2     int valeur;           // De 1      104
3     int tetes_de_boeuf;   // 1, 2, 3, 5 ou 7 t t e s
4 } Carte;
5
6 typedef struct {
7     Carte cartes[NB_MAX_CARTES_RANGE];
8     int nb_cartes;
9     int total_tetes;
10 } Rangee;
11
12 typedef struct {
13     int id;
14     char nom[TAILLE_NOM];
15     int score;
16     Carte main[NB_CARTES_MAIN];
17     int nb_cartes_main;
18     int est_robot;
19     int est_actif;
20 } Joueur;
```

Listing 9 – Structures de données principales

6.2. Calcul des têtes de bœuf

```
1 int calculer_tetes_carte(int valeur) {
2     if (valeur <= 0 || valeur > NB_CARTES_TOTAL) return 0;
3
4     if (valeur % 55 == 0) return 7;    // Multiple de 55
5     if (valeur % 11 == 0) return 5;    // Multiple de 11
6     if (valeur % 10 == 0) return 3;    // Multiple de 10
7     if (valeur % 5 == 0) return 2;     // Multiple de 5
8     return 1;                          // Toutes les autres
9 }
```

Listing 10 – Calcul des têtes selon les règles

6.3. Gestion des parties

```

1 typedef struct {
2     int id_partie;
3     Joueur* joueurs[MAX_JOUEURS];
4     int nb_joueurs;
5     int joueurs_actifs;
6
7     Rangee rangees[NB_RANGEES];
8     Carte pioche[NB_CARTES_TOTAL];
9     int index_pioche;
10
11     int manche_actuelle;
12     int tour_actuel;
13     int joueur_actif;
14
15     int partie_terminee;
16     int gagnant_id;
17 } Partie;

```

Listing 11 – Structure d’une partie

7. Journalisation et statistiques

7.1. Système de logging

Tous les événements importants sont enregistrés dans un fichier de log avec timestamp :

```

1 void ecrire_log(const char* fichier_log, const char* message) {
2     FILE* log_file = fopen(fichier_log, "a");
3     if (log_file == NULL) return;
4
5     time_t now = time(NULL);
6     struct tm* tm_info = localtime(&now);
7     char timestamp[20];
8     strftime(timestamp, sizeof(timestamp), "%Y-%m-%d_%H:%M:%S",
9               tm_info);
10
11     fprintf(log_file, "[%s]_%s\n", timestamp, message);
12     fclose(log_file);
13 }

```

Listing 12 – Fonction de journalisation

7.2. Exemple de log

[2024-12-15 14:30:25] Partie 1 démarrée avec 4 joueurs

[2024-12-15 14:31:10] Alice joue carte 15 (2 têtes)
[2024-12-15 14:31:15] Bob prend 7 têtes sur la rangée 2
[2024-12-15 14:45:30] Partie 1 terminée - Gagnant: Charlie

7.3. Génération de statistiques

Un script shell/awk analyse les logs pour produire des statistiques :

```
1 #!/bin/bash
2 # stats.sh - Génère des statistiques à partir des logs
3
4 LOG_FILE="logs/parties.log"
5
6 echo "=== STATISTIQUES DES PARTIES ==="
7 echo ""
8
9 # Nombre total de parties
10 echo "Parties jouées:"
11 grep -c "Partie.*d marrée" $LOG_FILE
12
13 # Joueurs par partie
14 echo ""
15 echo "Répartition des joueurs:"
16 awk '/d marrée avec/{print $NF " joueurs"}' $LOG_FILE | sort |
    uniq -c
17
18 # Scores moyens
19 echo ""
20 echo "Scores moyens par joueur:"
21 # Analyse des logs pour extraire les scores
```

Listing 13 – Script de statistiques

8. Tests et validation

8.1. Stratégie de test

Nous avons adopté une approche de tests unitaires pour chaque module :

1. Tests de la logique du jeu
2. Tests des communications
3. Tests des joueurs robots
4. Tests d'intégration

8.2. Tests unitaires

Chaque module possède ses propres fonctions de test :

```
1 void tester_logique_jeu(void) {
2     printf("\n===_TEST_DE_LA_LOGIQUE_DU_JEU_===\n");
3
4     // Cr er une partie test
5     Partie* partie_test = creer_partie(1);
6
7     // Ajouter des joueurs
8     Joueur* j1 = creer_joueur(1, "Alice", 0);
9     Joueur* j2 = creer_joueur(2, "Bob", 1);
10
11     // Tester la distribution des cartes
12     distribuer_cartes_partie(partie_test);
13
14     // Tester le jeu de cartes
15     if (j1->nb_cartes_main > 0) {
16         Carte carte = retirer_carte_main(j1, 0);
17         int tetes = jouer_carte_partie(partie_test, j1->id, carte);
18         printf("Alice_a_pris_%d_t tetes\n", tetes);
19     }
20
21     // Nettoyer
22     detruire_partie(partie_test);
23     printf("===_FIN_DES_TESTS_===\n");
24 }
```

Listing 14 – Tests dans jeu.c

9. Manuel d'utilisation

9.1. Compilation

```
1 # Compilation manuelle
2 gcc -o 6quiprend main.c jeu.c joueurs.c communication.c
   gestionnaire.c -lpthread
3
4 # Utilisation du Makefile
5 make clean
6 make
```

Listing 15 – Compilation du projet

Commande	Description
-serveur [PORT]	Lance le serveur (port par défaut : 8080)
-client IP PORT	Lance un client et se connecte au serveur
-client-local	Lance un client local avec interface FIFO
-demo	Lance une démonstration avec des robots
-test	Lance tous les tests unitaires
-test-jeu	Teste uniquement la logique du jeu
-test-joueurs	Teste les joueurs et robots
-test-comm	Teste les communications
-help	Affiche l'aide
-version	Affiche la version

TABLE 3 – Commandes disponibles

9.2. Commandes disponibles

9.3. Exemples d'utilisation

```

1 # 1. Lancer une démonstration
2 ./6quiprend --demo
3
4 # 2. Lancer le serveur
5 ./6quiprend --serveur 8080
6
7 # 3. Se connecter au serveur (dans un autre terminal)
8 ./6quiprend --client 127.0.0.1 8080
9
10 # 4. Jouer en local
11 ./6quiprend --client-local
12
13 # 5. Lancer tous les tests
14 ./6quiprend --test

```

Listing 16 – Exemples d'utilisation

10. Discussion et améliorations

10.1. Difficultés rencontrées

- Synchronisation fine entre les multiples processus
- Gestion des déconnexions réseau imprévues
- Debugging des conditions de course
- Conversion des données réseau (byte order)
- Gestion mémoire dans un environnement multi-processus

10.2. Contribution de l'IA générative

Nous avons utilisé ChatGPT pour :

- Générer des commentaires de documentation
- Proposer des structures de données optimisées
- Aider au debugging de problèmes complexes
- Générer des exemples de code pour des fonctionnalités spécifiques
- Améliorer la qualité générale du code

11. Conclusion

11.1. Bilan technique

Le projet "6 qui prend" en ligne a été développé avec succès en respectant toutes les contraintes imposées. L'application intègre :

- Une architecture client-serveur robuste
- Des communications inter-processus fiables (FIFOs, sémaphores)
- Un serveur réseau multi-clients avec TCP/IP
- Une implémentation complète des règles du jeu
- Des joueurs robots avec différents niveaux de difficulté
- Un système de journalisation et de statistiques

11.2. Perspectives

Le projet pourrait évoluer vers une plateforme de jeu en ligne complète avec classements, tournois et communauté de joueurs. Les compétences acquises sont directement transférables au développement d'applications distribuées professionnelles.

11.3. Code source

Il comprend :

- 5 fichiers source C
- 4 fichiers header
- Un Makefile pour la compilation
- Des scripts
- La documentation technique

Annexes

Annexe A : Fichiers du projet

```
src/  
  main.c          # Point d'entrée principal  
  jeu.c           # Logique du jeu  
  jeu.h           # Définitions des structures  
  joueurs.c       # Gestion des joueurs  
  joueurs.h       # Messages et stratégies  
  communication.c # FIFOs, sockets, sémaphores  
  communication.h # Prototypes des communications  
  gestionnaire.c   # Orchestrateur principal  
  gestionnaire.h   # Structures de gestion  
  Makefile        # Fichier de construction  
logs/  
  parties.log     # Fichier de journalisation  
scripts/  
  stats.awk       # Script de statistiques  
  install.sh  
README.md        # Documentation
```